
task-app 30日間カリキュラム

トラブルシューティングガイド

Next.js 15 / TypeScript / Prisma / tRPC

目次

よくあるエラー	5
一般的なエラーと対処法.....	18
デバッグのコツ	20
戻る	20
奥付.....	21

開発中に遭遇しやすいエラーと解決方法をまとめました。

次の3枚は完成時の画面です。途中の Day では、まだ作っていないボタンや機能があっても問題ありません。その日の完成画面は各 Day の教材で確認してください。

3枚は、同じアカウントで続けて開いた画面ではありません。ログイン画面はログイン前、ダッシュボードは一般ユーザー、タスク一覧は管理者の表示例です。管理者のサイドバーにだけ「ユーザー管理」が表示されるため、2枚のサイドバーは項目数が異なります。



図1: 正常起動時のログイン画面

トラブルシューティングガイド



図 2: 正常起動時のダッシュボード画面



図 3: 正常起動時のタスク一覧画面

よくあるエラー

Day 01-02: 環境構築

`node: command not found`

原因: Node.js がインストールされていません。

解決方法: Day 01 の OS 別手順で Node.js 22 系をインストールします。mise 利用時は `task-app` フォルダで `mise trust && mise install` を実行します。

`npm install` でエラー

原因: Node.js のバージョンが教材と異なる可能性があります。

解決方法: `node -v` で確認します。対応範囲は 22.12.0 以上・23.0.0 未満で、推奨版は 22.22.2 です。22.11 や 23.0.0 など範囲外の版では初期セットアップが停止するため、Day 01 の OS 別手順で対応範囲の版に変更します。mise 利用時は `mise trust && mise install` を実行します。

`Environment variable not found: DATABASE_URL`

原因: DB 接続に必要な環境変数がありません。

解決方法: `task-app` フォルダの `.env` に `DATABASE_URL` を設定します。

`prisma generate` でエラー

原因: スキーマの記述ミスなどがあります。

解決方法: エラー本文のファイル名と行番号を確認し、`schema.prisma` を教材のコードと見比べます。

PostgreSQLに接続できない

原因: DockerまたはPostgreSQLが起動していません。

解決方法: `task-app` フォルダで `docker compose up -d` を実行してコンテナを起動します。

別フォルダの DB と衝突した場合

Day 01 の初期セットアップで「同じ Compose project 名を別のフォルダが使用しています」または「既存 volume がこの作業フォルダの DB に属することを確認できません」と表示された場合の手順です。「DATABASE_URL が今回起動する教材用 DB と一致しません」と表示された場合も、接続先を確認します。同じポート番号を別の DB やアプリが使っている状態をポート競合と呼びます。「アプリ用 DB の起動に失敗しました」と表示された場合はポート競合のことが多いため、この場合も次の手順で使う番号を分けます。

原因: Docker Compose は関連するコンテナを **project(プロジェクト)** という名前でまとめます。通常はフォルダ名がその名前になります。別の場所にある2つの `task-app` も同じ名前になるため、既存の DB を使い回さないように初期セットアップを停止しています。Ubuntu を再インストールしたあとに以前の DB が残っている場合も停止します。volume(DB のデータを保存する Docker の領域)も project ごとに分けて使います。

解決方法: 今回使う project 名とポート番号を、次の手順で分けます。

1. 今回展開したフォルダを VS Code で開きます。

macOS と直接利用の Ubuntu では VS Code を起動し、**File** (**ファイル**)
 → **Open Folder...** (**フォルダーを開く...**) から、ホームフォルダ内の
workspace/task-app を開きます。Windows では Ubuntu のターミナルで次
 を実行します。左下に **WSL: Ubuntu** など、自分が使っている Ubuntu の名前が
 表示されることを確認してください。

```
cd ~/workspace/task-app
code .
```

Windows で **code: command not found** が出る場合や、左下に WSL の表示
 がない場合は、次を確認します。

Windows 側に VS Code^{*1} をインストールします。インストーラの **Add to PATH**
 にチェックを入れます。これはターミナルから **code** を呼び出すための設定です。す
 でにインストール済みでも、設定を忘れた場合はインストーラを開き直します。

Windows の VS Code で **Ctrl + Shift + X** を押して拡張機能の一覧を開
 きます。検索欄に **@id:ms-vscode-remote.remote-wsl** と入力します。

Microsoft の **WSL** を選び、**Install** (インストール) を押します。この拡張機能で
 Ubuntu 内のファイルを編集できます。**Enable** (有効にする) が出ている場合は、
 それを押します。

Ubuntu のターミナルを閉じ、Windows のスタートメニューから開き直します。次
 の 2 行を再実行します。

1. <https://code.visualstudio.com/download>

```
cd ~/workspace/task-app  
code .
```

初回は必要な部品のダウンロードを待ちます。左下に **WSL: Ubuntu** などの表示が出たら次へ進みます。表示されない場合は VS Code 公式の WSL 手順^{*2}の「From VS Code」を使います。VS Code で **F1** を押し、**WSL: Connect to WSL using Distro** を選びます。使っている Ubuntu を選び、**File** → **Open Folder...** から **~/workspace/task-app** を開いてください。

2. 左側のファイル一覧で **.env** を開きます。

初期セットアップは DB の確認前に **.env** を作成します。名前が似ている **.env.example** は設定の見本です。ここで編集するのは **.env** です。

3. **.env** の次の5つの設定をそろえます。

すでにある項目はその行を置き換え、まだない **COMPOSE_PROJECT_NAME** は1行追加します。ほかの設定は残します。自動で追加された **_TASKAPP_SCAFFOLD_DB_OWNER** は今回の DB を区別する印なので、変更せず残してください。

2. <https://code.visualstudio.com/docs/remote/wsl>

```
COMPOSE_PROJECT_NAME=taskapp-study-01

_DOCKER_COMPOSE_HOST_PORT_DB=36532

_DOCKER_COMPOSE_HOST_PORT_TEST_DB=36533

DATABASE_URL="postgresql://user:password@localhost:36532/taskapp"

TEST_DATABASE_URL="postgresql://user:password@localhost:36533/taskapp_test"
```

`COMPOSE_PROJECT_NAME` は今回のコンテナと volume に付ける project 名です。`36532` はアプリ用 DB、`36533` はテスト用 DB のポート番号です。接続先 URL にも対応する番号を入れます。名前だけを変えても、同じポートを 2 つの DB で使うことはできません。

4. 保存して、ターミナルの古い設定を解除します。

macOS では `Command + S`、Windows と Ubuntu では `Ctrl + S` で保存します。次のコマンドを Day 01 の作業用ターミナルで実行します。

```
unset COMPOSE_PROJECT_NAME

unset _DOCKER_COMPOSE_HOST_PORT_DB _DOCKER_COMPOSE_HOST_PORT_TEST_DB

unset DATABASE_URL TEST_DATABASE_URL
```

`unset` は現在のターミナルで設定した環境変数を解除するコマンドです。ターミナル側に古い設定があると `.env` より優先されるため、今回編集したファイルを使える状態にします。`.env` の内容は消えません。エラーが表示されず入力待ちに戻ったら、次へ進みます。

5. Day 01 の初期セットアップを再実行します。

```
cd ~/workspace/task-app
bash scripts/scaffold-from-scratch.sh
```

DB セットアップが完了しました。と表示されたら Day 01 の Step 3 へ戻ります。新しい DB が作られ、以前の project のコンテナとデータは残ります。この再実行は Day 01 の初期セットアップを中断した場合の手順です。Day 02 以降の自分のコードやデータを残す用途では使いません。

同じ project 名のエラーが続く場合は、`.env` の `taskapp-study-01` を `taskapp-study-02` へ変えて保存します。ポート競合が続く場合は、`_DOCKER_COMPOSE_HOST_PORT_DB` と `DATABASE_URL` のポートを `36534` にそろえます。`_DOCKER_COMPOSE_HOST_PORT_TEST_DB` と `TEST_DATABASE_URL` のポートは `36535` にそろえてから再実行します。

その番号も使用中なら、既存の DB やアプリはそのままにして、今回の番号を `36536` と `36537` へ変えます。使用中の番号が続く場合は2ずつ増やして試します。番号を変えるたびに、対応する URL のポートも変えて保存してください。ポート競合以外のエラーは番号を変えても解決しないため、表示された内容に対応する項目をこの付録から探します。初期セットアップが完了したあとは、後続手順でも自分の `.env` の番号を使います。

DB 所有印が複数ありますと表示された場合

原因: 初回セットアップを複数のターミナルで同時に実行すると、`.env` に DB を区別する印を重複して保存する場合があります。どの印を使うか自動では判断せず、DB を変更する前に停止します。

解決方法: 同時に実行したセットアップを止め、今回の DB を新しい project 名で作り直します。以前の DB は削除しません。

1. 実行中のセットアップを止めます。

セットアップを実行した各ターミナルを確認します。まだ処理が続いている場合は **Ctrl + C** で止めます。入力待ちに戻っている場合は、そのまま構いません。

2. **.env** を開きます。

VS Code で今回の **task-app** フォルダの **.env** を開きます。開き方は直前の「別フォルダの DB と衝突した場合」の手順1・2を参照してください。

3. 重複した印を削除して保存します。

_TASKAPP_SCAFFOLD_DB_OWNER= で始まる行をすべて削除します。ほかの設定は残します。macOS は **Command + S**、Windows と Ubuntu は **Ctrl + S** で保存します。

4. 新しい project 名で1回だけ実行します。

直前の「別フォルダの DB と衝突した場合」の手順3～5を実行します。project 名は今回まだ使っていない名前にします。**taskapp-study-01** を使った場合は **taskapp-study-02** にします。セットアップは1つのターミナルだけで実行してください。

セットアップが新しい印を自動で保存します。**DB セットアップが完了しました。** と表示されたら、Day 01 の Step 3 へ戻ります。

Day 03-04: Git・デプロイ

`git push` で認証エラー

原因: Day 03 で行った `gh auth login` の認証が終わっていないか、途中で切れています。

解決方法: `gh auth status` を実行して状態を読みます。

`You are not logged into any GitHub hosts` と出たら

`gh auth login` からやり直します。3 番目の質問

`Authenticate Git with your GitHub credentials?` は `Yes` にします (Day 03 Step 5 と同じ手順です)。教材では SSH 鍵や Personal Access Token は使いません。

Vercel デプロイでビルドエラー

原因: 原因はデプロイログの最初のエラーで分かります。後ろのエラーは、最初の失敗から続けて発生している場合があります。

解決方法: デプロイログを開き、最初のエラーを読みます。

`環境変数の検証に失敗しました` と出ている場合は、次の項目へ進みます。型エラーやパッケージのインストール失敗など、別のメッセージが最初に出ている場合は、そのファイル名やコマンドを先に直します。

`環境変数の検証に失敗しました`

原因: `.env` に必要な変数がないか、`JWT_SECRET` が 32 文字未満です。

解決方法: エラー本文に出ている変数名を `.env.example` と見比べて `.env` に追記します。Vercel では Project Settings の Environment Variables に `DATABASE_URL` と `JWT_SECRET` を登録します。

Day 05-08: 認証・ログイン

Day 25 のパスワード変更後など、後半でログインできなくなった場合もこの項目で確認します。

ログインできない

原因: 開発者ツールの Network タブを開いたままログインし、失敗したリクエストの Status と Response を読みます。リクエスト自体が失敗している場合は、

`npm run dev` を実行しているターミナルの最初のエラーも確認します。データベース接続などのエラーが出ている間は、seed を実行しても直りません。認証情報が違うと表示された場合は、入力したメールアドレスとパスワードを見直します。Day 25 でパスワードを変更したあとは、変更後のパスワードを使います。ユーザーの有無を確認める必要がある場合だけ、次の手順へ進みます。

`npm run db:seed` を実行する前に読んでください。このコマンドは初期データの 2 つのプロジェクトをいったん削除して作り直します。そのプロジェクトに自分で追加したタスクとコメントも一緒に消えます。それ以外のプロジェクトやユーザーは対象外です。実行前に消える対象の一覧が表示され、
`続行しますか？ [y/N]:` と確認されます。追加した内容を残したい場合は `n` で中止してください。

解決方法:

- 1) Day 25 で変更したパスワードを使います。
- 2) `task-app` フォルダで `npx prisma studio` を実行し、開いた画面の User テーブルでユーザーの有無を確認します。

3) ユーザーが存在する場合は `npm run db:seed` を実行しません。すでにいるユーザーのパスワードは入れ直されません。

4) ユーザーごと消えている場合だけ `npm run db:seed` を実行します。初期データの2プロジェクトと、その中のタスク・コメントは作り直され、自分で追加した内容は消えます。

JWT_SECRET エラー

原因: 環境変数 `JWT_SECRET` が未設定または32文字未満です。

解決方法: `.env` に32文字以上のランダム文字列を設定します。

ログイン後にリダイレクトされない

原因: Cookie を保存できていないか、画面移動の処理に問題がある可能性があります。

解決方法: 開発者ツールのApplicationタブでCookiesを開き、接続先のサイトにセッションCookieがあるか確認します。無ければDay 07のCookie設定と見比べます。保存されていればログイン成功後の画面移動処理を確認します。

ログインが必要です と表示される

原因: セッションが切れています。

解決方法: 再度ログインします。

Day 09-12: プロジェクト・メンバー管理

プロジェクト一覧が空

原因: 開発者ツールの Network タブでプロジェクト一覧のリクエストを開きます。エラーになっている場合は、Status と Response に表示された認証・権限・接続の問題です。成功して空の配列が返っている場合は、ログインユーザーの参加するプロジェクトがまだありません。

解決方法: リクエストが成功して空だった場合は、新規プロジェクトを作成します。

`npm run db:seed` は上の注意を読んでから実行してください。

メンバー追加できない

原因: OWNER/ADMIN 権限がありません。

解決方法: 自分の権限を確認します。権限がなければプロジェクトオーナーに依頼します。

このユーザーは既にプロジェクトのメンバーです

原因: すでにメンバーとして追加済みです。

解決方法: メンバー一覧を確認します。

Day 13-16: タスク管理

タスクが表示されない

原因: 開いているプロジェクトと画面のフィルターを確認します。次に開発者ツールの Network タブでタスク一覧のリクエストを開き、成功した空の結果なのか、認証・権限・接続のエラーなのかを分けます。

解決方法: フィルターを外しても成功した結果が空の場合は、そのプロジェクトにタスクがあるか、自分がメンバーに入っているかを確認します。リクエストが失敗している場合は、Responseのエラーを先に直します。

タスクの担当者を設定できない

原因: 担当者がプロジェクトメンバーではありません。

解決方法: まずメンバーとして追加してから担当者に設定します。

ステータス変更が反映されない

原因: 開発者ツールのNetworkタブでステータス変更のリクエストを確認します。失敗している場合は、Responseとターミナルに表示されたエラーが原因です。成功しているのに表示だけが古い場合は、一覧の控え(キャッシュ)が更新されていません。

解決方法: リクエストが成功している場合は、ステータス変更の `useMutation` の `onSuccess` に `utils.task.getAll.invalidate()` があるか確認します。リロード(WindowsとLinuxはCtrl+Shift+R、MacはCmd+Shift+R)で直る場合も、この呼び忘れを確認します。

Day 17-25: 応用機能

検索結果が0件

原因: 自分が関わるタスクまたはプロジェクトだけが対象です。

解決方法: 探しているタスクのプロジェクトにメンバーとして入っているか確認します。入っていれば検索キーワードとフィルター条件を見直します。

レポートにデータが表示されない

原因: 週次レポートは、対象ユーザーが担当する期間内の完了タスクを集計します。そのユーザーが現在参加しているプロジェクトのタスクだけが対象です。概要レポートは未完了タスクも集計しますが、アーカイブ済みプロジェクトとキャンセル済みタスクは対象外です。

解決方法: 週次レポートでは、担当者・完了日時・選択した期間・プロジェクトへの参加状況を確認します。表示のためだけに未完了のタスクを完了へ変更する必要はありません。概要レポートでは、参加中のプロジェクトがアーカイブされていないか、タスクがキャンセルされていないか確認します。

作業時間が保存されない

原因: 時間と分の合計が0のまま保存しようとしています。この場合は

1分以上入力してください と表示されます。

解決方法: 時間か分を入力し、合計を1分以上にして保存します。時間は0以上の整数、分は0～59の整数です。小数・負の数などを入力した場合は、入力欄のエラーに従って直します。送信中は保存が終わるまで待ちます。

Day 26-30: 仕上げ・公開

型エラー(`tsc --noEmit`)

原因: TypeScriptの型定義が不正です。

解決方法: エラーメッセージの行番号を確認して型を修正します。

Biomeのエラー

原因: コードスタイルに違反しています。

解決方法: `npm run lint:fix` で自動修正します。

一般的なエラーと対処法

Module not found

```
Module not found: Can't resolve '@component/ui/button'
```

原因: インポートパスが間違っているかファイルが存在しません。

解決: ファイルの存在を確認します。パスエイリアス(`@/`)が正しく設定されているか `tsconfig.json` を確認します。

Hydration Error

```
Error: Hydration failed because the initial UI  
does not match what was rendered on the server
```

原因: サーバーとブラウザ(クライアント)で異なるHTMLが生成されています。

解決: `Date.now()` や `Math.random()` のように実行するたびに変わる値を最初の描画で使っていないか確認します。 `'use client'` を付けても直りません。 `'use client'` を付けたコンポーネントもブラウザへ送る前にサーバー側で一度描画されます。そのためサーバーとブラウザで値がずれる状況は変わりません。変わる値は `useEffect` の中で計算して `useState` に入れます。画面へ出すのはブラウザでの描画からにします。最初の描画では `--` など固定の文字を表示しておくともサーバーとブラウザのHTMLが一致します。

HTMLタグの入れ子も確認します。たとえば `<p>` の中に `<div>` を置いている場合は、タグの構成を直します。この原因は `useEffect` へ処理を移しても解決しません。

TRPC Error: ログインが必要です

原因: 認証が必要なAPIに未認証でアクセスしています。

解決: ログイン画面からログインし直します。Cookieが正しく設定されているか確認します。

Prisma: Invalid `prisma.xxx.findMany()` invocation

原因: この見出しだけでは原因を特定できません。続くエラー本文に、引数の誤りや接続先の問題などが表示されます。

解決: エラー本文の最後まで確認します。引数名や型の誤りなら呼び出し元を教材と見比べます。スキーマを変更したのにクライアントへ反映していなかった場合は、`task-app` フォルダで `npx prisma generate` を実行します。接続エラーなら、この付録の「Day 01-02: 環境構築」へ戻ります。

Port 3000 is in use ... using available port 3001 instead.

原因: すでに別のプロセス(実行中のプログラム)がポート3000を使用しています。

解決: 同じ `task-app` フォルダの開発サーバーが3000番を使っているときはいま起動した方を `Ctrl+C` で止めます。ブラウザは3000番のまま開きます。別のアプリが3000番を使っているときは開発サーバーが空いている番号で起動します。このメッセージに出た番号をブラウザで開いてください。3000番のまま使いたい場合は、先に起動した開発サーバーのターミナルで `Ctrl+C` を押して終了します。見つか

らない場合は `lsof -i :3000` で使用中のプロセスを確認します。Windows では Day 01 と同じ Ubuntu のターミナルで実行してください。自分で起動した開発サーバーと確認できたものを終了してから起動し直します。

デバッグのコツ

1. ブラウザの開発者ツールを活用

- **Console** タブ: エラーメッセージの確認
- **Network** タブ: API リクエスト/レスポンスの確認
- **Application** タブ: Cookie の確認

2. サーバーログの確認

ターミナルで `npm run dev` を実行しているウィンドウのログを確認します。エラーの詳細な内容はここに表示されます。

3. エラーメッセージを検索

エラーメッセージをそのままコピーして Google で検索します。Stack Overflow や公式ドキュメントで解決策が見つかることも多いです。

4. 変更を小さく保つ

大きな変更を一度に行うとエラーの特定が難しいです。小さな変更 → 確認 → コミットを繰り返します。

戻る

- 目次: カリキュラム目次^{*3}
- 全体の地図: 学びのロードマップ^{*4}

奥付

タイトル	トラブルシューティングガイド
著者	磯貝光佑
版	第1版(2026年9月19日)

© 2026 磯貝光佑

本書は購入者個人の利用に限ります。本書に掲載されたコードの写経、および自分のプロジェクトへの転用・改変は自由です。本書(PDF・Markdown)および付属コードの再配布・転売・第三者との共有は、形態を問わず禁止します。

-
3. <https://drive.google.com/file/d/19g0cKLajEWi7s9LeWF804DB9j5kK1dt4/view?usp=drivesdk>
 4. https://drive.google.com/file/d/10py_p-6ry4RDlxBSyzPn4rtzZe74ND-T/view?usp=drivesdk